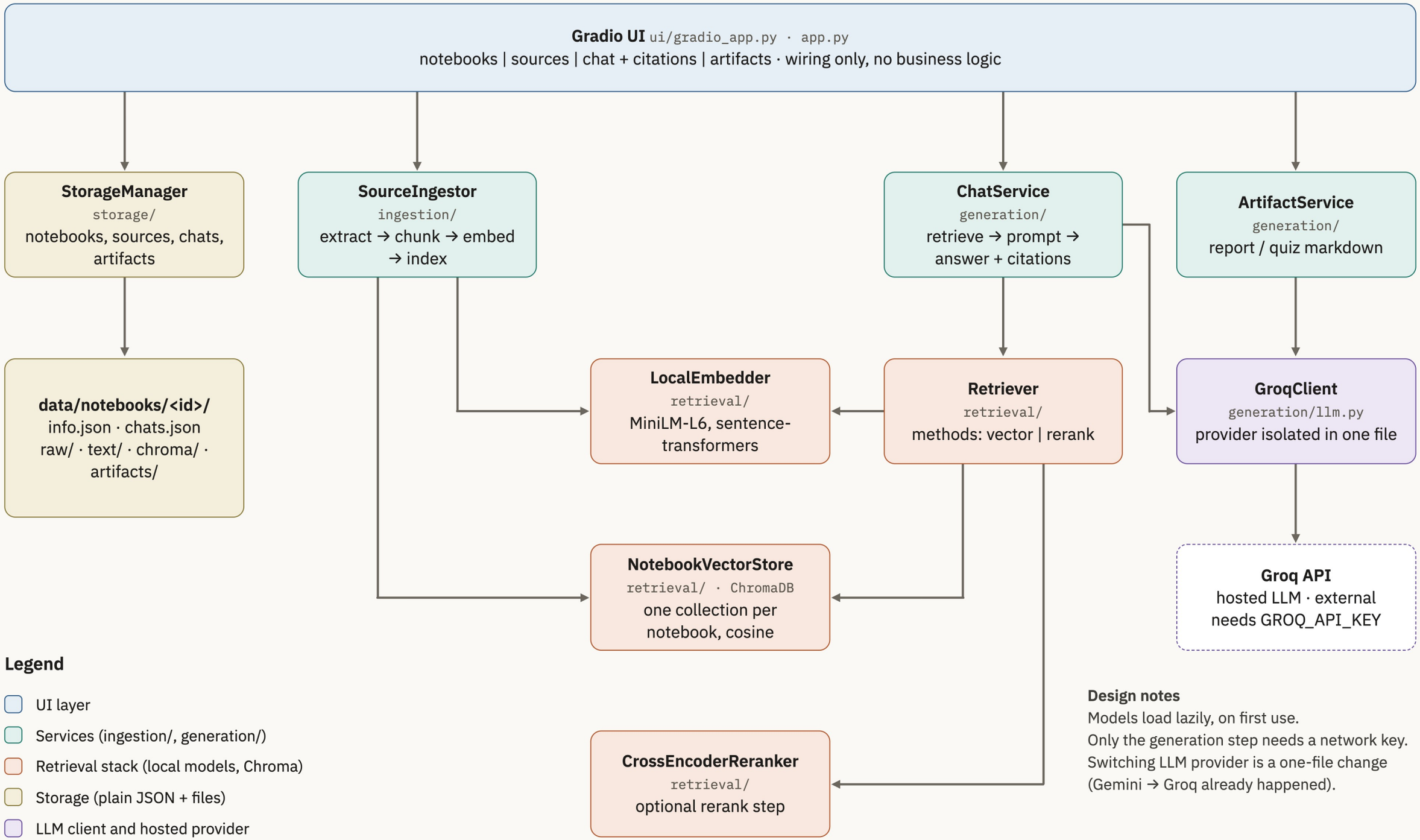
NotebookLM Clone · System architecture

Gradio UI over four services, a local retrieval stack and a hosted LLM. Every notebook is isolated: its own folder and its own Chroma collection.



Legend

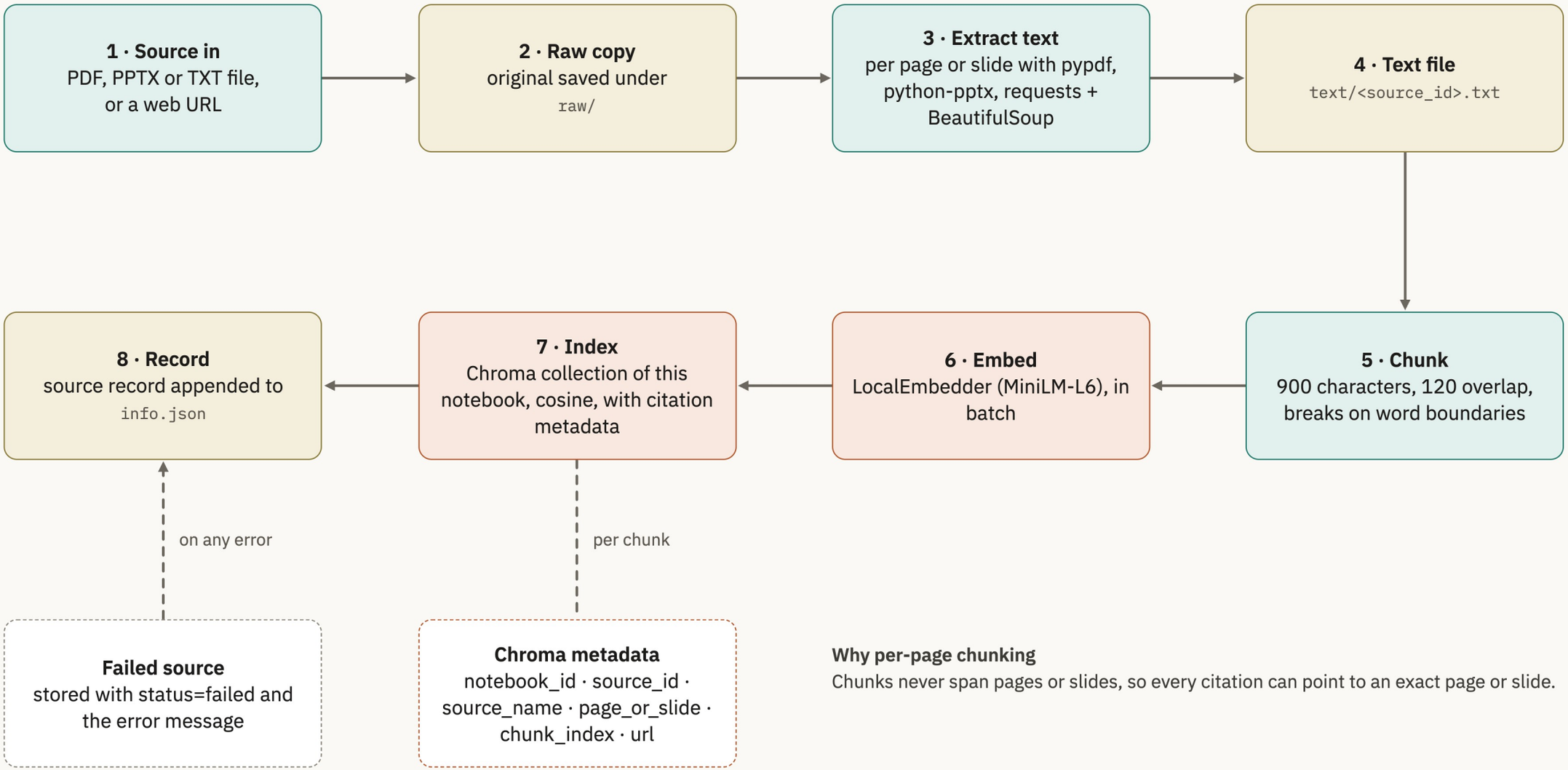
- UI layer
- Services (ingestion/, generation/)
- Retrieval stack (local models, Chroma)
- Storage (plain JSON + files)
- LLM client and hosted provider

Design notes

Models load lazily, on first use.
Only the generation step needs a network key.
Switching LLM provider is a one-file change (Gemini → Groq already happened).

Ingest flow

From an uploaded file or URL to searchable, citable chunks. Read left to right, then right to left on the second row.

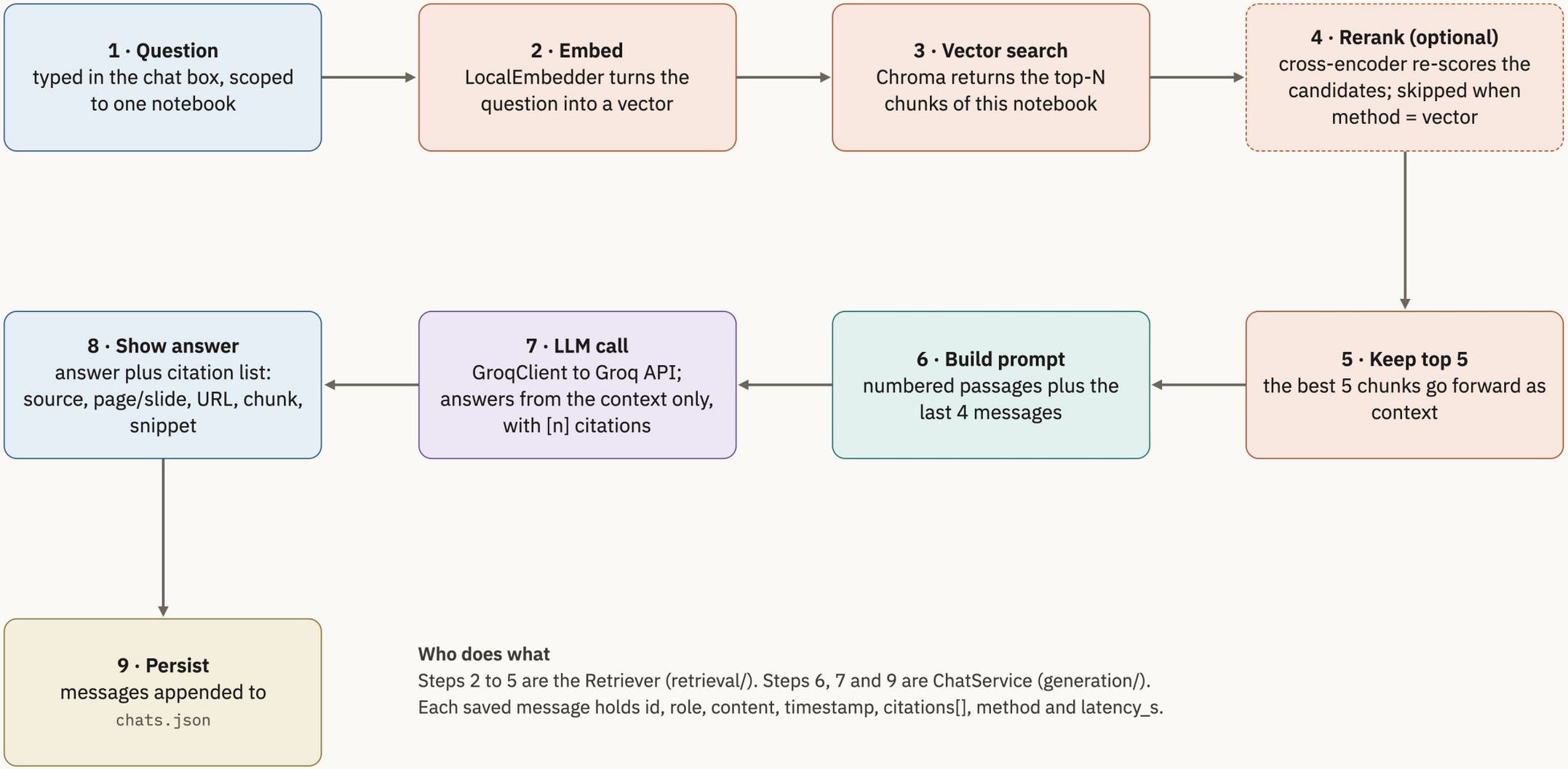


Why per-page chunking

Chunks never span pages or slides, so every citation can point to an exact page or slide.

Chat flow (RAG)

One question, grounded in the active notebook's chunks. Read left to right, then right to left on the second row.



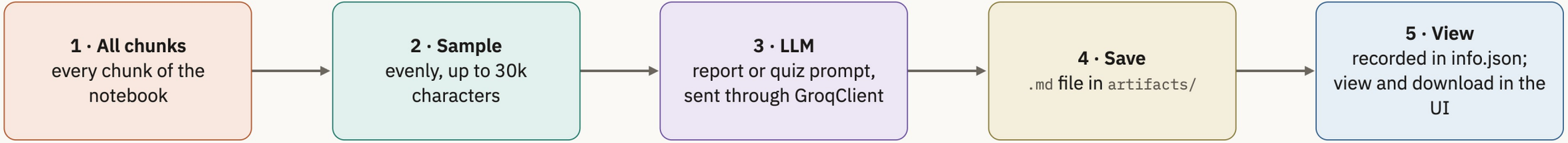
Who does what

Steps 2 to 5 are the Retriever (retrieval/). Steps 6, 7 and 9 are ChatService (generation/).
Each saved message holds id, role, content, timestamp, citations[], method and latency_s.

Artifacts and data model

How reports and quizzes are generated, and what a notebook folder stores on disk.

Artifact generation (ArtifactService)



Notebook data model

info.json
id, name, created_at, updated_at
sources[]: id, type, name, path, status, metadata
artifacts[]: id, type, filename, path, created_at

chats.json
a list of messages, each with
id, role, content, timestamp, citations[], method, latency_s

Notebook isolation
Each notebook has its own folder and its own Chroma collection, so retrieval can never cross notebooks.

Deployment

A push to main runs the tests, builds the image and deploys it to Cloud Run. Secrets stay out of the repo.

